《压缩解压技术》实验指导书

目录

《压缩解压技术》实验指导书

目录
1. 压缩技术概述
    1.1 为什么需要压缩?
    1.2 压缩的分类
    1.3 压缩算法分类
2. Huffman编码原理
    2.1 核心思想
    2.2 哈夫曼树的构造
    2.3 Huffman编码生成
    2.4 解码过程
    2.5 文件压缩中的词频统计
    2.6 编码后的比特打包
    2.7 解码所需信息存储
3. Huffman编码工程实现（CHuffman类）
    3.1 类结构总览
    3.2 压缩流程（5 步曲）
    3.3 解压流程
    3.4 压缩文件格式（CHuffman）
4. LZ77算法原理
    4.1 核心思想
    4.2 滑动窗口模型
    4.3 编码示例
    4.4 解码过程
    4.5 滑动窗口大小限制
5. LZ77工程实现（CLZ77类）
    5.1 类结构总览
    5.2 压缩流程
    5.3 解压流程
    5.4 LZ77压缩文件格式
6. 两种算法对比与混合方案
    6.1 Huffman vs LZ77
    6.2 混合方案：DEFLATE（LZ77 + Huffman）
    6.3 在备份软件中的选择
7. 课堂思考题
    Q1：Huffman编码为什么必须统计Byte频率而不是字符频率?
    Q2：Huffman编码的码表（dict）是 vector，每个码字是字符串如 "110"，编码时直接拼接 codecBuffer += dict[word]。这样会不会导致内存爆炸?
    Q3：LZ77中，如果字典中没有匹配，`searchLongestMatch` 返回的 `longestMatchLen` 是多少? `rcd` 如何编码?
    Q4：LZ77解压时，从字典复制数据可能发生重叠（如 index=1, len=10），代码如何正确处理?
    Q5：为什么 LZ77 的 `FILE_BUFFER_SIZE` 要远大于 `DICT_SIZE + BUFF_SIZE`?
    Q6：如果要实现 DEFLATE（LZ77 + Huffman），在现有框架下如何修改?
8. 总结
    核心知识点
    代码设计亮点

扩展方向

1. 压缩技术概述

1.1 为什么需要压缩?

- 节省存储空间
- 减少网络传输时间
- 本质：**消除数据中的冗余信息**

1.2 压缩的分类

类别	特点	代表算法	适用场景
无损压缩	解压后与原始数据完全一致	Huffman、LZ77、DEFLATE	文本、程序、备份数据
有损压缩	丢弃部分人类不敏感的信息	JPEG、MP3、MPEG	图像、音频、视频

> 本课程的数据备份软件需要**无损压缩**，确保备份数据可完全还原。

1.3 压缩算法分类

类别	原理	代表算法
基于统计	利用符号出现频率不均，高频短码	Huffman、Shannon-Fano
基于字典	利用数据重复片段，用引用替代	LZ77、LZ78
混合方法	结合两者	DEFLATE（LZ77 + Huffman）

2. Huffman编码原理

2.1 核心思想

出现频率高的字符用短编码，频率低的用长编码，从而实现整体编码长度最短。

2.2 哈夫曼树的构造

给定权值集合 {7, 5, 2, 4}，构造过程如下：

```
步骤1：森林中每棵树只有一个节点
   [7]   [5]   [2]   [4]

步骤2：选出最小的两个 2 和 4，合并为 6
   [7]   [5]      [6]
                 /   \
               [2]   [4]

步骤3：选出最小的两个 5 和 6，合并为 11
```

```
   [7]          [11]
             /      \
          [5]       [6]
                  /    \
                [2]    [4]

步骤4：合并 7 和 11，得到根 18
            [18]
          /      \
       [7]       [11]
               /      \
            [5]       [6]
                    /    \
                  [2]    [4]

最终哈夫曼树构造完成。
```

2.3 Huffman编码生成

规则：左分支标 `0`，右分支标 `1`，从根到叶子的路径即为该字符编码。

以字符串 `"ABACCDA"` 为例：

- 字符频率：A:3, B:1, C:2, D:1
- 构造的哈夫曼树（一种可能形态）：

```
           [7]
         /      \
      A(3)      [4]
              /      \
           C(2)     [2]
                  /    \
                B(1)  D(1)
```

- 编码结果：
    - A: `0`
    - C: `10`
    - B: `110`
    - D: `111`

原始字符串 `ABACCDA` → **压缩后比特串** `0 110 0 10 10 111 0` = `0110010101110`（13 bits）

原始用 ASCII 需要 7×8=56 bits，压缩率 13/56 ≈ 23.2%。

2.4 解码过程

从根开始，按比特串走：

- 遇到 `0` 往左，遇到 `1` 往右
- 走到叶子即输出该字符，然后回到根继续

例如 `0110010101110`：

```
0 → A
1→1→0 → B
0 → A
1→0 → C
1→0 → C
1→1→1 → D
0 → A
```

输出 `ABACCDA`，完美还原。

2.5 文件压缩中的词频统计

关键认知：文件在内存中是二进制字节流，因此统计单位是 **Byte（8 bits）**，而不是字母或汉字。

```
#define WORD_NUM 256   // 0~255 共 256 个可能的字节值

struct WordCount {
    long int table[WORD_NUM];  // table[byte] = 该字节出现次数
};

while ((ret = sfFile.readN(&word, sizeof(uint8_t))) > 0) {
    wc.table[word]++;   // 统计每个字节值出现的次数
}
```

2.6 编码后的比特打包

警告：不要用 `char` 数组存储 `'0'` / `'1'` 字符串！那样会使文件膨胀（每个比特占用一个字节）。

正确做法：将比特串按 8 位一组打包成真正的字节（位操作）。

```
class BitVec {
public:
    BitVec(const std::string &str) {  // str 长度必须为 8
        value = 0;
        int shift = 7;
        for (char c : str) {
            if (c == '1') value |= (1u << shift);
            shift--;
        }
    }
    uint8_t to_uint8_t() { return value; }
};
```

填充问题：Huffman 编码后的比特串长度不一定是 8 的整数倍，最后需要补 `0` 对齐。

```cpp
if (codecBuffer.size() % 8 != 0) {
    paddingNum = 8 - (codecBuffer.size() % 8);
    codecBuffer.append(paddingNum, '0');   // 补零
}
// 必须将 paddingNum 写入压缩文件，解压时去掉多余的填充位
```

2.7 解码所需信息存储

压缩文件中除了编码数据，还必须存储解码所需的辅助信息。有三种方案：

方案	存储内容	优点	缺点
编码对照表	字符 → 编码（含长度）	解码直接查表	表大小不固定，占空间
词频表	每个字符的出现次数	可重新建树，统一格式	需要重建树，多一次计算
静态树结构	树的序列化表示	解码无需建树	存储复杂

工程代码采用方案二（词频表），因为 256 个字节的频率表大小固定（`256 × sizeof(long)`），便于读写。

3. Huffman编码工程实现（CHuffman类）

3.1 类结构总览

```cpp
class CHuffman : public ICompressionAlgorithm {
public:
    int compress(std::string file, std::string targetFile) override;
    int decompress(std::string file, std::string targetFile) override;

protected:
    struct WordCount { long int table[WORD_NUM]; };
    struct HuffmanNode { uint8_t data; long freq; HuffmanNode *left, *right; };

    int doCompress(Utils::CSimpleFile& sfFile, Utils::CSimpleFile& targetFile);
    int doDecompress(Utils::CSimpleFile& sfFile, Utils::CSimpleFile& targetFile);

    HuffmanNode* createHuffmanTree(const WordCount& wc);
    void destroyHuffmanTree(HuffmanNode* root);
    void generateCodeDict(HuffmanNode* root, std::string code,
std::vector<std::string>& dict);
    int huffmanCoding(Utils::CSimpleFile& sfFile, Utils::CSimpleFile& targetFile,
                      std::vector<std::string>& dict);
    int huffmanDecoding(Utils::CSimpleFile& sfFile, Utils::CSimpleFile& targetFile,
                        HuffmanNode* root, uint8_t paddingNum);

    class BitVec { ... };  // 8位比特 ↔ 字节 转换工具
};
```

3.2 压缩流程（5 步曲）

```
┌──────────────────────────────────────────────────────────────────┐
│                      CHuffman::compress                          │
├──────────────────────────────────────────────────────────────────┤
│ 1. compressionCheck()              // 源文件存在性/可读性检查          │
│ 2. 创建目标文件，写入 commonHeaderStruct（算法标识 HUFFMAN）          │
│ 3. doCompress()                    // 核心压缩逻辑                  │
│    ├── 统计词频 (WordCount)                                        │
│    ├── 写入词频表到目标文件 (sizeof(WordCount) 固定)                 │
│    ├── createHuffmanTree()         // 构造哈夫曼树                  │
│    ├── generateCodeDict()          // 生成编码字典 (vector<string>) │
│    ├── destroyHuffmanTree()        // 释放树内存                    │
│    └── huffmanCoding()             // 二次读取源文件，逐字节编码写入  │
│         ├── 预写入 paddingNum 占位（稍后回填）                       │
│         ├── 读取源文件，逐字节查 dict 拼接比特串                     │
│         ├── 每满 CODEC_BUFFER_SIZE 位，打包成字节写盘                │
│         ├── 末尾不足 8 位补零，记录 paddingNum                      │
│         └── seek 回填 paddingNum 到文件头后固定位置                  │
│ 4. updateFileCRC64()               // 更新校验和                    │
└──────────────────────────────────────────────────────────────────┘
```

关键代码段:

词频统计与写入:

```
WordCount wc;
memset(&wc, 0, sizeof(wc));
uint8_t word;
while ((ret = sfFile.readN(&word, 1)) > 0) {
    wc.table[word]++;
}
targetFile.writeN(&wc, sizeof(wc));   // 词频表放在文件头
```

构造哈夫曼树（使用优先队列）：

```
HuffmanNode* createHuffmanTree(const WordCount& wc) {
    // 小顶堆比较器：频率小的优先
    struct NodeCompare {
        bool operator()(HuffmanNode*& a, HuffmanNode*& b) {
            return a->freq > b->freq;  // 注意：返回 true 表示 a 优先级低于 b
        }
    };
    priority_queue<HuffmanNode*, vector<HuffmanNode*>, NodeCompare> pq;
    for (int i = 0; i < 256; i++) {
        if (wc.table[i] > 0)
            pq.push(new HuffmanNode((uint8_t)i, wc.table[i]));
    }
    while (pq.size() > 1) {
        auto left = pq.top(); pq.pop();
        auto right = pq.top(); pq.pop();
        auto parent = new HuffmanNode(left->data, left->freq + right->freq);
```

```
        parent->left = left; parent->right = right;
        pq.push(parent);
    }
    return pq.empty() ? nullptr : pq.top();
}
```

生成编码字典（先序遍历）：

```
void generateCodeDict(HuffmanNode* root, string code, vector<string>& dict) {
    if (!root) return;
    if (!root->left && !root->right) {
        dict[root->data] = code;   // 叶子节点：存储编码
        return;
    }
    generateCodeDict(root->left, code + "0", dict);
    generateCodeDict(root->right, code + "1", dict);
}
```

编码与位打包：

```
int huffmanCoding(...) {
    // 先占位写 paddingNum
    uint8_t paddingNum = 0;
    targetFile.writeN(&paddingNum, 1);

    string codecBuffer;  // 临时比特串
    while (读取一个字节 word) {
        codecBuffer += dict[word];
        if (codecBuffer.size() > CODEC_BUFFER_SIZE) {
            for (int i = 0; i < CODEC_BUFFER_SIZE; i += 8) {
                BitVec bv(codecBuffer.substr(i, 8));
                targetFile.writeN(&bv.to_uint8_t(), 1);
            }
            codecBuffer = codecBuffer.substr(CODEC_BUFFER_SIZE);
        }
    }
    // 末尾补零对齐
    if (codecBuffer.size() % 8 != 0) {
        paddingNum = 8 - (codecBuffer.size() % 8);
        codecBuffer.append(paddingNum, '0');
    }
    // 写剩余字节
    for (...) { ... }

    // 回填 paddingNum
    targetFile.seekFile(sizeof(commonHeaderStruct) + sizeof(WordCount), SEEK_SET);
    targetFile.writeN(&paddingNum, 1);
}
```

3.3 解压流程

```
┌──────────────────────────────────────────────────────────────┐
│                    CHuffman::decompress                      │
├──────────────────────────────────────────────────────────────┤
│ 1. decompressionCheck()          // 源文件校验 + 公共头校验      │
│ 2. 跳过 commonHeaderStruct                                    │
│ 3. doDecompress()                                            │
│    ├── 读取词频表 (WordCount)                                  │
│    ├── createHuffmanTree(wc)    // 重建哈夫曼树                │
│    ├── 读取 paddingNum                                        │
│    ├── huffmanDecoding()                                     │
│    │   ├── 循环读取压缩数据中的每一个字节，转为8位比特串          │
│    │   ├── 将比特串追加到 codecBuffer                         │
│    │   ├── 遍历 codecBuffer 中的比特，从根开始走 0/1           │
│    │   ├── 走到叶子则输出该字节，回到根继续                    │
│    │   └── 边解码边将已处理的比特从 buffer 中移除（优化）       │
│    ├── 最后去掉 paddingNum 个填充位，解码剩余比特               │
│    └── destroyHuffmanTree()                                  │
└──────────────────────────────────────────────────────────────┘
```

解码核心逻辑：

```
int huffmanDecoding(..., HuffmanNode* root, uint8_t paddingNum) {
    string codecBuffer;
    uint8_t word;
    while (sfFile.readN(&word, 1) > 0) {
        BitVec bv(word);
        codecBuffer += bv.to_string();  // 转成 "0101..." 字符串
        // 边解码边处理，避免 buffer 无限增长
        HuffmanNode* ptr = root;
        for (size_t i = 0; i < codecBuffer.size(); i++) {
            ptr = (codecBuffer[i] == '0') ? ptr->left : ptr->right;
            if (!ptr->left && !ptr->right) {
                targetFile.writeN(&ptr->data, 1);
                ptr = root;
                // 如果剩余比特不够一个最长编码，则前移并跳出
                if (codecBuffer.size() - 1 - i < CODE_LENGTH) {
                    codecBuffer = codecBuffer.substr(i + 1);
                    break;
                }
            }
        }
    }
    // 去掉填充位，解码剩余
    for (int i = 0; i < paddingNum; i++) codecBuffer.pop_back();
    // 继续解码剩余比特...
}
```

3.4 压缩文件格式（CHuffman）

```
┌──────────────────────────────────────────────────────┐
| commonHeaderStruct                                   |
|  · Algorithm = HUFFMAN_ALGORITHM                     |
|  · CRC64 = 校验值                                    |
|  · offset = sizeof(commonHeaderStruct)               |
├──────────────────────────────────────────────────────┤
| WordCount (256 × 8 = 2048 字节)                      |
|   table[0..255] 各字节出现次数                       |
├──────────────────────────────────────────────────────┤
| uint8_t paddingNum (占1字节，实际写入位置)           |
├──────────────────────────────────────────────────────┤
| 编码后的二进制数据 (每个字节 8 位)                   |
|   (最后一个字节含填充位)                             |
└──────────────────────────────────────────────────────┘
```

注意：`paddingNum` 在文件中的位置在词频表之后、编码数据之前。代码中先占位写入0，编码结束后再回填真实值。

4. LZ77算法原理

4.1 核心思想

利用数据中**重复出现的字符串**：当遇到重复片段时，用 **（距离，长度，下一个字符）** 的三元组代替原始数据。

- **距离**：从当前位置向前回溯多少个字节。
- **长度**：匹配到的重复字符串长度。
- **下一个字符**：匹配结束后紧跟的那个新字符（用于处理边界）。

4.2 滑动窗口模型

编码器维护一个**滑动窗口**，分为两部分：

```
[ ← 字典区域 (已编码历史) ] [ 前向缓冲区 (待编码) ]
            ↑                          ↑
        光标(当前位置)
```

- **字典区域**：已编码过的数据，作为匹配参考。
- **前向缓冲区**：即将编码的数据，在其中寻找与字典的最长匹配。

4.3 编码示例

原始数据：`abcabcabc`

编码过程（假设窗口足够大）：

1. 光标在 `a`，字典为空 → 编码 `(0, 0, 'a')`，输出 `a`。
2. 光标在 `b`，字典 `a` 无匹配 → 编码 `(0, 0, 'b')`，输出 `b`。
3. 光标在 `c`，字典 `ab` 无匹配 → 编码 `(0, 0, 'c')`，输出 `c`。

4. 光标在 `a`（第4个字符），字典中有 `abc`，向前看，匹配到 `abc`（长度3）→ 编码 `(3, 3, 'a')`，表示“往前3个字节，抄3个字节，再跟一个 `a`”。
5. 以此类推...

压缩效果：用三元组替代了长串重复数据。

4.4 解码过程

解压时维护同样的滑动窗口，按顺序读取三元组：

- 如果是 `(0, 0, c)`，直接输出 `c`。
- 如果是 `(距离, 长度, 下一个)`，从当前窗口向前 `距离` 位置开始，复制 `长度` 个字符，再追加 `下一个` 字符。

解码是确定性的，不需要额外存储字典。

4.5 滑动窗口大小限制

实际实现中，窗口大小有限（如 64KB），因为：

- 内存有限
- 过大的窗口导致匹配搜索开销大
- 距离字段需要定长编码

5. LZ77工程实现（CLZ77类）

5.1 类结构总览

```
class CLZ77 : public ICompressionAlgorithm {
public:
    int compress(...) override;
    int decompress(...) override;

protected:
    // 编码记录：距离（2字节）+ 长度（1字节）+ 下一字符（1字节） = 4字节
    struct LZ77Record {
        uint16_t index;   // 距离（回退字节数）
        uint8_t len;      // 匹配长度
        char next;        // 后续字符
    };

    int doCompress(Utils::CSimpleFile& sfFile, Utils::CSimpleFile& targetFile);
    int doDecompress(Utils::CSimpleFile& sfFile, Utils::CSimpleFile& targetFile);

    // 在字典中搜索与前向缓冲区的最长匹配
    const char* searchLongestMatch(const char* dictBegin, const char* buffBegin,
                                   const char* buffEnd, int& longestMatchLen);
};
```

常量定义：

```
#define MAX_DICT_SIZE 65535   // 2^16 - 1, index 最大 65535
#define MAX_BUFF_SIZE 255     // 2^8 - 1, len 最大 255
#define FILE_BUFFER_SIZE (1024 * 128)  // 128KB 文件缓冲区
```

5.2 压缩流程

```
┌──────────────────────────────────────────────────────────────────────┐
|                      CLZ77::doCompress                               |
├──────────────────────────────────────────────────────────────────────┤
| 1. 从源文件读取 FILE_BUFFER_SIZE 字节到内存缓冲区 fileBuffBegin    |
| 2. 设置滑动窗口指针：                                                 |
|    · buffBegin = 前向缓冲区起始（初始为 fileBuffBegin）             |
|    · dictBegin = 字典起始（初始为 fileBuffBegin，大小为0）          |
|    · fileBuffEnd = 当前文件缓冲区有效末尾                           |
| 3. while (buffBegin < fileBuffEnd) {                                 |
|      // 在字典中搜索与前向缓冲区的最长匹配                          |
|      matchLen, matchBegin = searchLongestMatch(                      |
|           dictBegin, buffBegin, buffEnd-1);                          |
|      // 构造 LZ77Record                                              |
|      rcd.index = buffBegin - matchBegin;                             |
|      rcd.len = matchLen;                                             |
|      rcd.next = *(buffBegin + matchLen);                             |
|      // 写入目标文件                                                 |
|      targetFile.writeN(&rcd, sizeof(rcd));                           |
|      // 前移窗口: buffBegin += matchLen + 1                          |
|      // 若窗口触及缓冲区尾部且文件未读完，则刷新缓冲区:              |
|      //   将 dictBegin 到 fileBuffEnd 的数据移到缓冲区头部          |
|      //   继续读文件填充剩余空间                                     |
|  }                                                                   |
└──────────────────────────────────────────────────────────────────────┘
```

searchLongestMatch 算法:

```
const char* searchLongestMatch(const char* dictBegin, const char* buffBegin,
                               const char* buffEnd, int& longestMatchLen) {
    longestMatchLen = 0;
    const char* longestMatchBegin = buffBegin;

    int len1 = buffEnd - dictBegin - 1;
    int len2 = buffEnd - buffBegin;

    // 从字典每个位置开始尝试匹配
    for (int i = 0; i <= len1 - len2; i++) {
        int x = i, y = 0;
        while (y < len2 && dictBegin[x] == buffBegin[y]) {
            x++; y++;
        }
        if (y > longestMatchLen) {
            longestMatchLen = y;
            longestMatchBegin = dictBegin + i;
        }
```

```
            if (y == len2) break;  // 已完全匹配，提前结束
        }
        return longestMatchBegin;
    }
```

缓冲区刷新机制：
当 `buffBegin` 前移使前向缓冲区剩余不足 `BUFF_SIZE` 时，需从文件读入更多数据。但为保证滑动窗口连续性，需将字典末尾的部分数据保留在缓冲区开头。

```
if (buffSize() < BUFF_SIZE && !fileToEnd) {
    int remained = fileBuffEnd - dictBegin();  // 字典 + 前向缓冲区中未处理的数据
    memcpy(fileBuffBegin, dictBegin(), remained);
    buffBegin = fileBuffBegin + dictSize();    // 重新定位前向缓冲区头
    // 读取新数据填充剩余空间
    ret = sfFile.readN(fileBuffBegin + remained, FILE_BUFFER_SIZE - remained);
    fileBuffEnd = fileBuffBegin + remained + ret;
}
```

5.3 解压流程

```
┌──────────────────────────────────────────────────────────────────┐
|                      CLZ77::doDecompress                         |
├──────────────────────────────────────────────────────────────────┤
| 1. 初始化内存缓冲区 fileBuffBegin (大小 FILE_BUFFER_SIZE)        |
|    buffBegin = fileBuffBegin，字典大小为 0                        |
| 2. while (读取一个 LZ77Record 成功) {                             |
|      // 解码：从字典中复制 len 个字符到 buffBegin                 |
|      for (i = 0; i < rcd.len; i++)                               |
|          buffBegin[i] = buffBegin[-(rcd.index - i)];             |
|      buffBegin[rcd.len] = rcd.next;                              |
|      // 前移窗口: buffBegin += rcd.len + 1                       |
|      // 如果缓冲区剩余空间不足，将字典之前已解码的数据刷盘       |
|      if (buffSize() < BUFF_SIZE) {                               |
|          targetFile.writeN(fileBuffBegin, dictBegin() - ...);    |
|          memcpy(fileBuffBegin, dictBegin(), dictSize());         |
|          buffBegin = fileBuffBegin + dictSize();                 |
|      }                                                           |
|  }                                                               |
| 3. 最后将缓冲区中所有数据刷盘                                    |
└──────────────────────────────────────────────────────────────────┘
```

关键点：解码时，从字典复制数据时可能存在**重叠**（如 `index` 较小，长度较长），但代码按顺序逐个字节复制，能正确处理重叠情况。

5.4 LZ77压缩文件格式

```
┌──────────────────────────────────────────────────────┐
| commonHeaderStruct                                   |
|  · Algorithm = LZ77_ALGORITHM                        |
|  · CRC64 = 校验值                                    |
|  · offset = sizeof(commonHeaderStruct)               |
├──────────────────────────────────────────────────────┤
| LZ77Record 序列 (每个 4 字节)                        |
|  [index:2B][len:1B][next:1B]                         |
|  [index:2B][len:1B][next:1B]                         |
|  ...                                                 |
|  (无特殊结束标记，读到 EOF 即结束)                   |
└──────────────────────────────────────────────────────┘
```

不需要词频表等辅助信息，因为解码所需信息（滑动窗口）可由记录本身重建。

6. 两种算法对比与混合方案

6.1 Huffman vs LZ77

维度	Huffman	LZ77
基本原理	统计字符频率，变长编码	查找重复字符串，用引用替代
压缩率	对重复性高的文件效果一般	对重复性高的文件效果显著
适用场景	符号分布不均匀的文本	有大段重复数据的文件
编码/解码速度	较快（查表）	较慢（需匹配搜索）
内存占用	小（树 + 字典表）	较大（滑动窗口缓冲区）
是否需要预知数据	需要先统计词频（两遍扫描）	流式压缩，一遍扫描
压缩文件头	需存储词频表或树结构	无需额外辅助信息

6.2 混合方案：DEFLATE（LZ77 + Huffman）

- **思路**：先用 LZ77 查找重复，输出（距离，长度）对；再对距离和长度用 Huffman 编码。
- **代表**：ZIP、gzip、PNG 等广泛使用的格式。
- **优点**：结合两种算法的优势，压缩率更高。

6.3 在备份软件中的选择

要求	推荐算法
备份文档、代码	Huffman（效果尚可，实现简单）
备份虚拟机镜像、数据库	LZ77（重复数据多）

要求	推荐算法
追求极致压缩率	DEFLATE（但实现复杂）

7. 课堂思考题

Q1：Huffman编码为什么必须统计Byte频率而不是字符频率?

答案：因为文件在内存和磁盘中是以二进制字节流存储的，压缩工具不关心文件的文本编码（ASCII/UTF-8/GBK等）。按字节统计可适用于任何类型的文件（文本、图片、程序等），具有通用性。

Q2：Huffman编码的码表（dict）是 vector，每个码字是字符串如 "110"，编码时直接拼接 codecBuffer += dict[word]。这样会不会导致内存爆炸?

答案：代码使用 `CODEC_BUFFER_SIZE` 控制 buffer 大小（8192位），每当 buffer 超过该阈值，就立即打包成字节写入文件，然后将剩余部分前移。因此内存占用是可控的。

Q3：LZ77中，如果字典中没有匹配，`searchLongestMatch` 返回的 `longestMatchLen` 是多少？`rcd` 如何编码?

答案：无匹配时 `longestMatchLen = 0`，`rcd.index` 也为 0（因为 `longestMatchBegin = buffBegin`），编码为 `(0, 0, next)`。解码时遇到 `len=0` 且 `index=0`，直接输出 `next`。

Q4：LZ77解压时，从字典复制数据可能发生重叠（如 index=1, len=10），代码如何正确处理?

答案：代码用 `for (i=0; i<rcd.len; i++) buffBegin[i] = buffBegin[-(rcd.index - i)];`。当 `index=1` 时，依次复制 `buffBegin[-1]`、`buffBegin[-2]`、...，但由于写入的位置在不断前移，第二次复制时 `buffBegin[-1]` 已经是刚写入的新字符，从而实现了重复扩展（如将 "ab" 扩展为 "ababab..."）。这正是 LZ77 的语义。

Q5：为什么 LZ77 的 `FILE_BUFFER_SIZE` 要远大于 `DICT_SIZE + BUFF_SIZE`?

答案：因为缓冲区刷新时采用**非循环缓冲区**设计（将剩余数据移动到头部），如果缓冲区太小，频繁的 `memcpy` 会降低效率。较大的缓冲区可以减少刷新次数，提高整体吞吐量。

Q6：如果要实现 DEFLATE（LZ77 + Huffman），在现有框架下如何修改?

答案：

1. 先将 LZ77 输出的 `(index, len, next)` 三元组中的 `index` 和 `len` 分别用 Huffman 编码。
2. 构建两个独立的 Huffman 树：一个用于距离值，一个用于长度值。
3. 压缩文件头部需要存储两棵树（或对应词频表）。
4. 解码时先解码出 `(index, len, next)`，再用 LZ77 的解码逻辑恢复原始数据。

8. 总结

核心知识点

1. **Huffman编码**：
    - 基于频率统计，高频短码
    - 构造哈夫曼树（优先队列）
    - 生成编码字典（递归先序遍历）
    - 位打包 + 填充处理
    - 压缩文件需存储词频表
2. **LZ77算法**：
    - 基于滑动窗口和重复字符串匹配
    - 编码为 `(距离，长度，下一个字符)` 三元组
    - 解码时重建窗口，流式处理
    - 无需额外辅助信息
3. **工程要点**：
    - 使用二进制读写，避免文本模式干扰
    - 位操作打包/解包
    - 缓冲区管理（避免频繁 I/O）
    - 文件头设计（算法标识、校验和）

代码设计亮点

- ✅ 统一接口 `ICompressionAlgorithm`，便于替换算法
- ✅ Huffman 使用优先队列简化建树
- ✅ 位向量类 `BitVec` 封装比特 ↔ 字节转换
- ✅ LZ77 使用非循环缓冲区，内存管理清晰
- ✅ 完善的错误检查和资源释放

扩展方向

- 🔧 实现 DEFLATE 混合算法
- 🔧 支持多线程并行压缩（分块处理）
- 🔧 自适应算法选择（根据文件类型自动选择 Huffman 或 LZ77）
- 🔧 流式压缩（边读边压，无需两次扫描）

> 📖 **推荐阅读**：
>
> - Huffman, D.A. (1952). "A Method for the Construction of Minimum-Redundancy Codes"
> - Ziv, J.; Lempel, A. (1977). "A Universal Algorithm for Sequential Data Compression"
> - DEFLATE Compressed Data Format Specification (RFC 1951)